# HOME-SHIELD

Public Tool Documentation
Home IoT Exposure Validator

**Platform: macOS / Linux | Language: Python 3.8+ | Interface: CLI | License: MIT**

# 1. About HOME-SHIELD

HOME-SHIELD is a defensive Python command-line tool that measures home and small-office/home-office network exposure.

It helps users determine whether a network change actually reduced what is visible or reachable from a specific network vantage point. Core idea: run a baseline scan, apply a hardening change, run a second scan, compare the two runs, and generate a human-readable HTML report.

| Category | Description |
|---|---|
| **Primary use case** | Validate whether segmentation, firewall rules, or service restrictions changed observable exposure. |
| **Target environment** | Home networks, SOHO networks, student labs, and homelabs. |
| **Primary outputs** | Structured JSON/CSV files, before/after diff data, and HTML reports. |
| **Main audience** | Technical home users, students, reviewers, and contributors. |

# 2. Supported Platforms and Requirements

| Requirement | Details |
|---|---|
| **Supported platforms** | macOS and Linux. |
| **Not fully tested** | Windows is not currently covered by this guide. |
| **Python version** | Python 3.8 or newer. |
| **Required package** | jinja2>=3.0 for HTML report generation. |
| **Optional package** | tshark for optional PCAP parsing. |
| **Network requirement** | A connected local network interface such as en0, eth0, or wlan0. |

# 3. Installation

The current public installation path is based on the repository README. Use this workflow for the most up-to-date setup process.

## 3.1 Install from the repository

```
git clone https://github.com/aanajmi/HOME-SHIELD.git
cd HOME-SHIELD
pip install -e .
```

If pip is not available as a standalone command, use Python to call pip directly:

```
python3 -m pip install -e .
```

### 3.2 Verify the tool starts

```
homeshield --help
```

You can also run the project directly from the repository folder:

```
python homeshield_main.py --help
```

### 3.3 Dependency fallback

If report generation fails because Jinja2 is missing, install it manually:

```
pip install jinja2
```

## 4. Quick Start Workflow

The standard HOME-SHIELD workflow is: Measure -> Harden -> Re-measure -> Compare -> Report.

| Step | Action | Description |
|---|---|---|
| 1 | Measure baseline | Run the first scan before changing the network. |
| 2 | Harden | Apply one controlled change, such as segmentation or a firewall rule. |
| 3 | Re-measure | Run the same scan again with a new label. |
| 4 | Compare | Use diff to compare the two run.json files. |
| 5 | Report | Generate a readable HTML report for review. |

### 4.1 Find your active interface

```
# macOS
route -n get default | grep interface

# Linux
ip link show
```

### 4.2 Run a baseline measurement

```
homeshield measure --label baseline_iot_run1 --vantage iot --
iface en0 \
    --rounds 5 --interval 10 --listen 4 --ports
80,443,445,554,1883
```

### 4.3 Apply one controlled hardening change

Examples include adjusting a firewall rule, changing VLAN policy, enabling client isolation, disabling an unnecessary service, or testing a reversible route change. Make one change at a time so the before/after comparison remains meaningful.

### 4.4 Run the second measurement

```
homeshield measure --label hardened_iot_run1 --vantage iot --
iface en0 \
    --rounds 5 --interval 10 --listen 4 --ports
80,443,445,554,1883
```

### 4.5 Compare and generate the report

```
homeshield diff --before outputs/baseline_iot_run1/run.json \
    --after outputs/hardened_iot_run1/run.json --out
outputs/diff.json

homeshield report --diff outputs/diff.json --out
reports/before_after.html
```

## 5. Command Reference

| Command | Purpose | Example |
|---|---|---|
| **measure** | Collects discovery and reachability results. | homeshield measure --label baseline ... |
| **diff** | Compares two run.json files. | homeshield diff --before ... --after ... |
| **report** | Generates HTML report from diff. | homeshield report --diff ... |
| **pcap-parse** | Parses PCAP for offline evidence. | homeshield pcap-parse --pcap capture.pcap |
| **--help** | Displays available commands. | homeshield --help |

## 6. Output Files and Reports

| File | Purpose |
|---|---|
| **run.json** | Full structured record of a measurement run. |
| **discovery.csv** | Discovery results, including mDNS and SSDP responders. |
| **reachability.csv** | TCP reachability results for tested hosts and ports. |
| **diff.json** | Before/after deltas and exposure score changes. |

| | |
|---|---|
| **HTML report** | Human-readable report for review or documentation. |

## 7. Scoring Model

The score starts at 100. Higher scores mean less observed exposure. Penalties are subtracted for discovery responders and reachable services.

| Exposure Item | Score Impact | Why It Matters |
|---|---|---|
| **Base score** | 100 | Starting point before exposure penalties. |
| **Unique mDNS responder** | -2 each | Indicates local service visibility. |
| **Unique SSDP responder** | -2 each | Indicates UPnP-style discovery visibility. |
| **OPEN TCP port** | -5 each | Indicates a reachable service. |
| **Newly OPEN port** | -8 extra | Flags possible regression or unintended exposure. |

## 8. Optional PCAP Parsing

PCAP parsing is optional and intended for users who want additional evidence from packet captures. Most users can ignore this feature at first.

```
homeshield pcap-parse --pcap evidence/pcaps/capture.pcap
```

## 9. Troubleshooting

| Issue | Recommended Check |
|---|---|
| **homeshield --help fails** | Confirm you are inside the repository folder and ran pip install -e . |
| **python command not found** | Use python3 instead of python on macOS or Linux. |
| **Wrong interface** | Run route -n get default \| grep interface (macOS) or ip link show (Linux). |
| **Report generation fails** | Install Jinja2 (pip install jinja2). |

## 10. Project Structure

```
HOME-SHIELD/
├── homeshield/          # source code
├── tests/               # test files
```

```
├── homeshield_main.py   # main entry point
├── setup.py             # packaging / install metadata
├── README.md
├── LICENSE
└── .gitignore
```

| Module | Responsibility |
|---|---|
| **commands/** | Command workflows. |
| **discovery/** | mDNS and SSDP discovery logic. |
| **reachability/** | TCP connect checks. |
| **diff/** | Comparison and scoring logic. |
| **report/** | HTML report generation. |
| **pcap/** | Optional PCAP parsing. |
| **utils/** | Logging, output, and helper logic. |

## 11. Safety, Scope, and Limitations

| Limit | Meaning |
|---|---|
| **Not a vulnerability scanner** | HOME-SHIELD does not exploit devices, validate credentials, or inspect firmware. |
| **Vantage-dependent results** | Results describe what is visible/reachable from the scanning machine. |
| **Discovery can vary** | mDNS/SSDP responders appear/disappear depending on network state. |
| **Reachable != vulnerable** | An open port indicates exposure, not necessarily compromise. |

## 12. Future Work

| Area | Planned Improvement |
|---|---|
| **Additional protocols** | Add more discovery and management protocol support. |
| **Improved visualization** | Add charts, exposure trends, and per-device timelines. |
| **Windows validation** | Evaluate future Windows support. |

## Appendix A. Quick Command Summary

| Task | Command |
|---|---|
| **Show help** | homeshield --help |
| **Baseline run** | homeshield measure --label baseline_run --vantage iot --iface en0 ... |
| **Compare** | homeshield diff --before ... --after ... --out outputs/diff.json |
| **Report** | homeshield report --diff outputs/diff.json --out reports/before_after.html |